



**MODUL PEMROGRAMAN MOBILE
CIM 430**

**MODUL PERTEMUAN 2
KONSEP DASAR ARSITEKTUR DAN PEMROGRAMAN *FLUTTER***

**DISUSUN OLEH :
7174 – SAWALI WAHYU, S.KOM, M.KOM**

Universitas
Esa Unggul

**UNIVERSITAS ESA UNGGUL
FAKULTAS ILMU KOMPUTER
TAHUN 2021**

KONSEP DASAR ARSITEKTUR DAN PEMROGRAMAN FLUTTER

A. Kemampuan Akhir Yang Diharapkan

Setelah mempelajari modul ini, diharapkan mahasiswa mampu :

1. Mahasiswa Mampu Memahami Arsitektur Pemrograman Flutter.
2. Mahasiswa Mampu Menggunakan *Tools* / Aplikasi Untuk Membuat Android dengan Pemrograman Flutter.
3. Mahasiswa Mampu Membuat Aplikasi Android dengan Pemrograman Flutter.

B. *Material Presentation*

- a) Konsep Dasar Pemrograman *Flutter*
- b) Arsitektur Pemrograman *Flutter*
- c) Konsep *Hybrid Apps* dengan Pemrograman *Dart*
- d) Praktikum : Membuat Aplikasi Android Sederhana



BASIC CONCEPTS OF ARCHITECTURE AND FLUTTER PROGRAMMING

1. Konsep Dasar Pemrograman *Flutter*



Gambar 1. Flutter Concept

Flutter adalah SDK untuk pengembangan aplikasi mobile yang dikembangkan oleh Google. Sama seperti *react native*, *framework* ini dapat digunakan untuk membuat atau mengembangkan aplikasi *mobile* yang dapat berjalan pada device iOS dan Android. Dibuat menggunakan bahasa C, C++, Dart dan Skia. pada *framework* ini semua kodenya di *compile* dalam kode native (Android NDK, LLVM, AOT-compiled) tanpa ada *intrepeter* pada prosesnya sehingga proses *compile*-nya menjadi lebih cepat. Dari segi penulisan kodenya, Flutter sangat berbeda dari *react native* dan lebih cenderung mendekati Java Android.

Versi pertama Flutter dikenal sebagai "Sky" dan berjalan pada sistem operasi Android. Diresmikan pada perhelatan *Dart developer summit* tahun 2015, dengan tujuan untuk mampu merender grafis secara konsisten pada 120 bingkai per detik. Komponen utama Flutter termasuk :

1) Flutter engine

Flutter engine, ditulis terutama dengan bahasa pemrograman C++, memberikan dukungan rendering tingkat rendah menggunakan *library* grafik Skia milik Google. Selain itu, *flutter engine* juga berinteraksi dengan perkakas pengembangan perangkat lunak (*SDK*) spesifik-serambi (*platform-specific*) seperti yang disediakan oleh Android dan iOS.

2) Foundation library

Foundation library, ditulis dengan bahasa pemrograman Dart, menyediakan fungsi dan *class-class* dasar yang digunakan untuk membangun aplikasi menggunakan Flutter, seperti *API* untuk berkomunikasi dengan *engine*.

3) Widget spesifik desain

Framework Flutter berisi dua set widget yang disesuaikan dengan bahasa desain tertentu. Widget *Material Design* menerapkan bahasa desain Google dengan nama yang sama, sedangkan widget 'Cupertino' meniru desain iOS milik Apple.

Kerangka kerja Flutter menawarkan fitur-fitur berikut kepada pengembang :

- a) Kerangka kerja modern dan reaktif.
- b) Menggunakan bahasa pemrograman Dart dan sangat mudah dipelajari.
- c) Perkembangan yang cepat.
- d) Antarmuka pengguna yang indah dan lancar.
- e) Katalog widget besar.
- f) Menjalankan UI yang sama untuk beberapa platform.
- g) Aplikasi berkinerja tinggi.

Begitu banyak kerangka yang bisa digunakan untuk mengembangkan aplikasi lintas platform, seperti React Native, Nativescript, dan Fuse. Namun yang membedakannya adalah, Flutter tidak menggunakan Webview maupun widget bawaan, Flutter punya mesin render sendiri untuk menampilkan widgetnya, hal ini menguntungkan developer yang ingin memiliki tampilan UI unik yang konsisten pada semua perangkat karena tidak bergantung pada widget bawaan OEM.

Kinerja Flutter yang tinggi ini tentunya didukung oleh berbagai teknologi terbaik. Flutter dibuat dengan C, C++, Dart, Skia untuk mesin render 2D, Mojo IPC, dan Blink untuk sistem render. Cara kerja Flutter pada platform Android

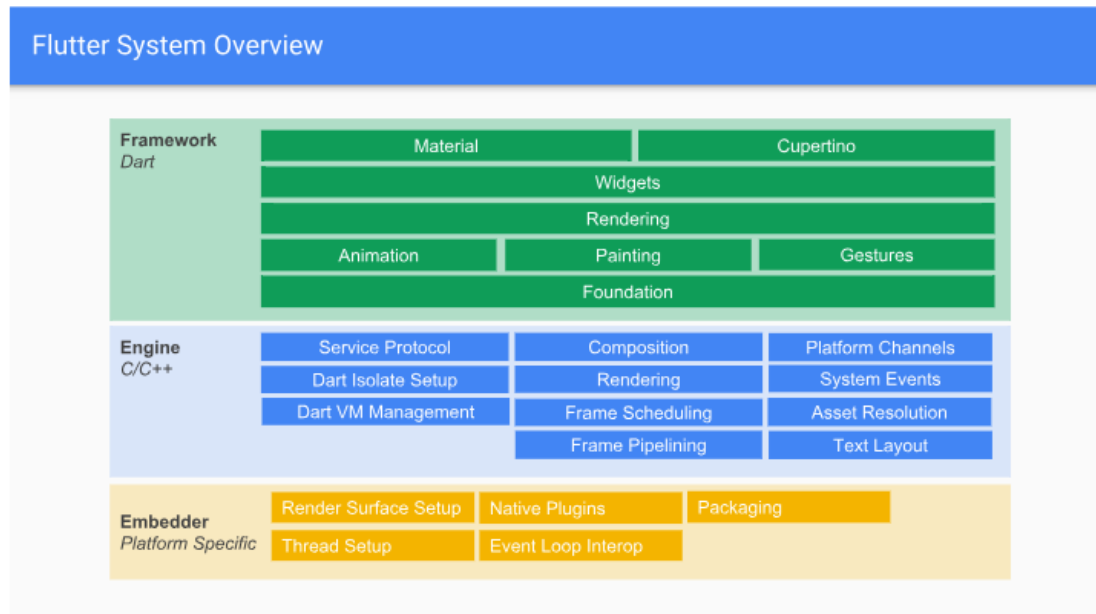
yaitu, kode C/C++ dikompilasi menggunakan Android NDK, sebagian besar kerangka dan kode aplikasi dijalankan dalam bentuk kode native yang dikompilasi oleh Dart compiler. Sedangkan pada platform iOS, kode dikompilasi dengan LLVM dan aplikasi dijalankan dengan kumpulan instruksi native tanpa interpreter.

Flutter dapat berjalan pada sistem operasi Android 4.1 atau lebih tinggi dan iOS 8 atau lebih tinggi; dan dapat dijalankan pada perangkat asli maupun Android emulator, serta iOS simulator. Semua developer pastinya menginginkan kinerja aplikasi yang paling optimal. Flutter menjanjikan kinerja yang sempurna, aplikasi yang dibangun dengan Flutter dapat berjalan secara konstan pada tingkat 60 frame per detik hingga 120fps. Alasan utama mengapa Flutter tidak menggunakan widget bawaan yaitu, Flutter tidak ingin kinerjanya dibatasi dan bergantung pada kinerja widget bawaan sehingga aplikasi kualitas tinggi dapat dihasilkan.

Flutter sudah dipaketkan dengan kode pengaksesan platform service dan API, seperti sensor, penyimpanan lokal dan lain sebagainya. Namun, untuk menghindari kendala, pengaksesan API untuk platform yang spesifik, dapat digunakan dengan plugin untuk integrasi. Jika ingin melakukan interoperabilitas dengan kode yang sudah ada seperti Java, Objective-C, atau Swift, bisa diaktifkan dengan melintaskan pesan secara fleksibel dengan *PlatformMessageChannel*.

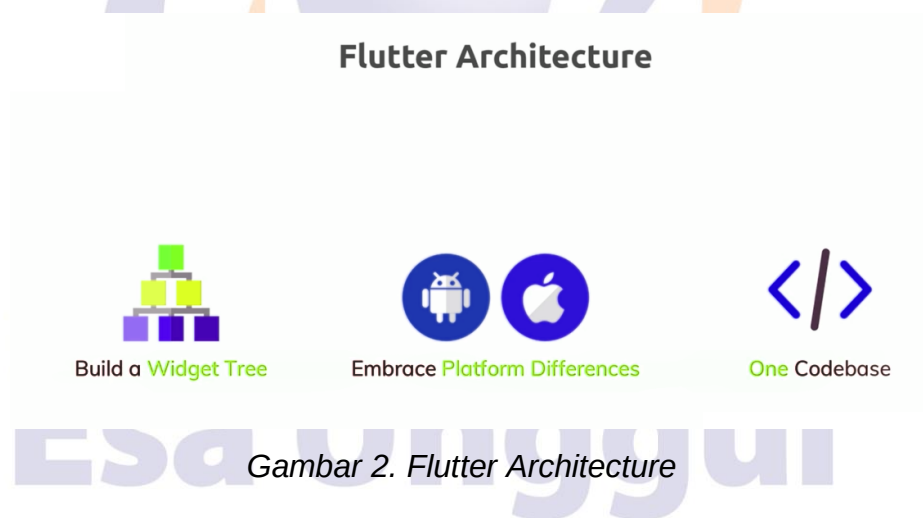
2. Arsitektur Pemrograman Flutter

Flutter menggabungkan Framework *Dart* dengan engine berperforma tinggi yang mengimplementasikan pustaka-pustaka inti Flutter termasuk animasi, grafis, file, network I/O dan dukungan aksesibilitas, arsitektur plugin, juga runtime Dart dan *toolchain* untuk melakukan proses development, *compiling* hingga running aplikasi Flutter. Engine flutter menggunakan teknologi Core, Skia, pustaka render grafis 2D, dan juga bahasa pemrograman Dart.



Gambar 1. Flutter Framework

Arsitektur Flutter



Gambar 2. Flutter Architecture

Everything's a widget. Semuanya adalah widget. Begitulah deskripsi singkat untuk menjelaskan konsep arsitektur dari Flutter. Widget adalah blok paling dasar dari aplikasi Flutter. Widget adalah komponen yang bersifat immutable dan menjadi bagian dari antarmuka pengguna. Tidak seperti framework lain yang memisahkan antara views, views controllers, layout dan properti lain. Flutter memiliki sebuah kesatuan object yang konsisten yang disebut widget.

Widget dapat terdiri dari,

- 1) Elemen terstruktur (seperti button atau menu)
- 2) Elemen stylistic (seperti font atau color scheme)
- 3) Aspek dari layout (seperti padding)

Meskipun Flutter menggunakan satu code base untuk aplikasi iOS dan Android. Namun tidak menutup kemungkinan developer dapat memisahkan programnya untuk iOS dan Android jika diperlukan sehingga anda dapat membuat program yang berbeda untuk sistem operasi iOS dan Android.

Kelebihan Flutter

Selain flutter ada banyak framework yang memungkinkan Developer membangun aplikasi *Mobile cross-platform*. Beberapa diantaranya adalah *React Native, Xamarin, Nativescript, dsb.*

Diantara semua alternatif tersebut, Flutter memiliki keunggulan sebagai berikut ;

- 1) Dart Bridge, size aplikasi memang lebih besar namun kinerjanya lebih cepat daripada React Native yang menggunakan Javascript bridge.
- 2) Support dari IDE (Android Studio, IntelliJ idea, VisualStudio Code).
- 3) Serba widget, dapat dikatakan bahwa *widget* adalah *basic building block* untuk membuat UI aplikasi Flutter.

3. Konsep *Hybrid Apps* dengan Pemrograman *Dart*

Bahasa pemrograman Dart merupakan bahasa pemrograman *general-purpose* yang dirancang oleh Lars Bak dan Kasper Lund. Bahasa pemrograman ini dikembangkan sebagai bahasa pemrograman aplikasi yang dapat dengan mudah untuk dipelajari dan disebarkan. Bahasa pemrograman besutan Google ini dapat digunakan untuk mengembangkan berbagai macam platform termasuk di dalamnya adalah *web*, aplikasi *mobile*, *server*, dan perangkat yang mengusung teknologi Internet of Things. Bahasa pemrograman tersebut dapat digunakan untuk mengembangkan aplikasi untuk dijalankan

pada berbagai macam peramban modern. Dart juga dapat digunakan untuk mengembangkan aplikasi dari *codebase* tunggal menjadi aplikasi Android maupun iOS. Bahasa pemrograman Dart dapat digunakan secara bebas oleh para developer, karena bahasa ini dirilis secara *open-source* oleh Google di bawah lisensi BSD. Bahasa pemrograman Dart merupakan bahasa pemrograman berbasis *class* dan berorientasi terhadap objek dengan menggunakan sitaks bahasa pemrograman C.

Bahasa ini dikenalkan oleh Google sebagai pengganti bahasa pemrograman JavaScript, akan tetapi secara opsional bahasa ini dapat dikompilasi ke dalam JavaScript dengan menggunakan *dart2js compiler*. Sedikit berbeda dengan bahasa pemrograman JavaScript yang bertipe statis, bahasa pemrograman Dart merupakan bahasa pemrograman bertipe dinamis.

Adapun kelebihan bahasa pemrograman Dart antara lain adalah sebagai berikut :

a) Fleksibel

Seperti yang dijelaskan sebelumnya, salah satu kelebihan bahasa pemrograman Dart adalah bahasa pemrograman tersebut termasuk ke dalam bahasa pemrograman bertipe dinamis. Bahasa pemrograman ini dapat dikompilasi ke dalam bahasa pemrograman JavaScript dengan *compiler* yang sudah disertakan di dalamnya.

Bahasa pemrograman ini dikembangkan untuk mudah digunakan dalam pengembangan, sesuai dengan pengembangan aplikasi modern, dan memiliki implementasi berkinerja tinggi. Bahkan, bahasa pemrograman ini dapat digunakan juga sebelum dikompilasi.

Dart VM menawarkan kemampuan untuk menjalankan secara langsung kode sumber tanpa perlu dikompilasi terlebih dulu. Bahasa pemrograman ini juga dapat langsung digunakan pada peramban Chrome tanpa perlu dikompilasi.

Bahasa pemrograman Dart mendukung banyak arsitektur, termasuk di dalamnya IA-32, X64, MIPS, ARMv5TE, ARMv6, ARMv7, dan arsitektur ARM64. Bahasa 31

b) Berdiri Sendiri

Pemrograman ini mendukung secara *native* pengembangan aplikasi *mobile* untuk ke dua platform Android dan iOS.

Kelebihan bahasa pemrograman Dart lainnya adalah ketersediaan SDK yang dilengkapi dengan berbagai macam *tools* pengembangan. Salah satu *tool*-nya adalah Dart VM, dimana *tool* tersebut akan membantu para developer untuk menjalankan kode dalam lingkungan tampilan *command line*.

Selain itu, dalam SDK tersebut juga terdapat *dart2js compiler* yang dapat digunakan untuk mengkompilasi Dart ke dalam bahasa pemrograman JavaScript. SDK tersebut juga dilengkapi dengan manajer paket yang disebut dengan *pup*, yang dapat digunakan untuk menggunakan kode pihak ketiga atau berbagi kodingan.

c) *Concurency*

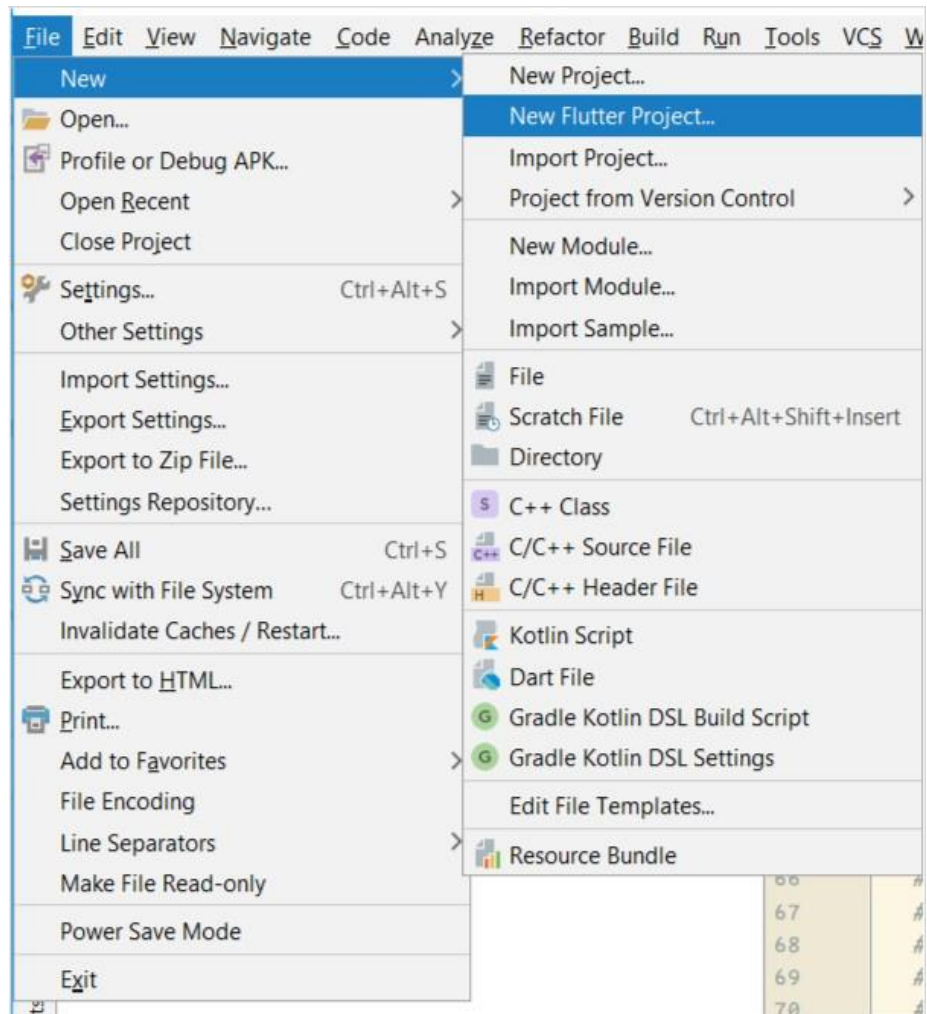
Bahasa pemrograman Dart memiliki kelebihan dengan adanya konstruksi nyata dari *concurrency* dan paralelisme. Kelebihan bahasa pemrograman Dart ini ditawarkan dengan bentuk Dart Isolates. Dengan adanya Dart Isolates, program-program akan terisolasi untuk bekerja secara independen tanpa adanya pembagian memori, akan tetapi tetap terdapat komunikasi diantaranya. Setiap program Dart menggunakan setidaknya satu buah isolasi.

4. Praktikum : Membuat Aplikasi Android Sederhana

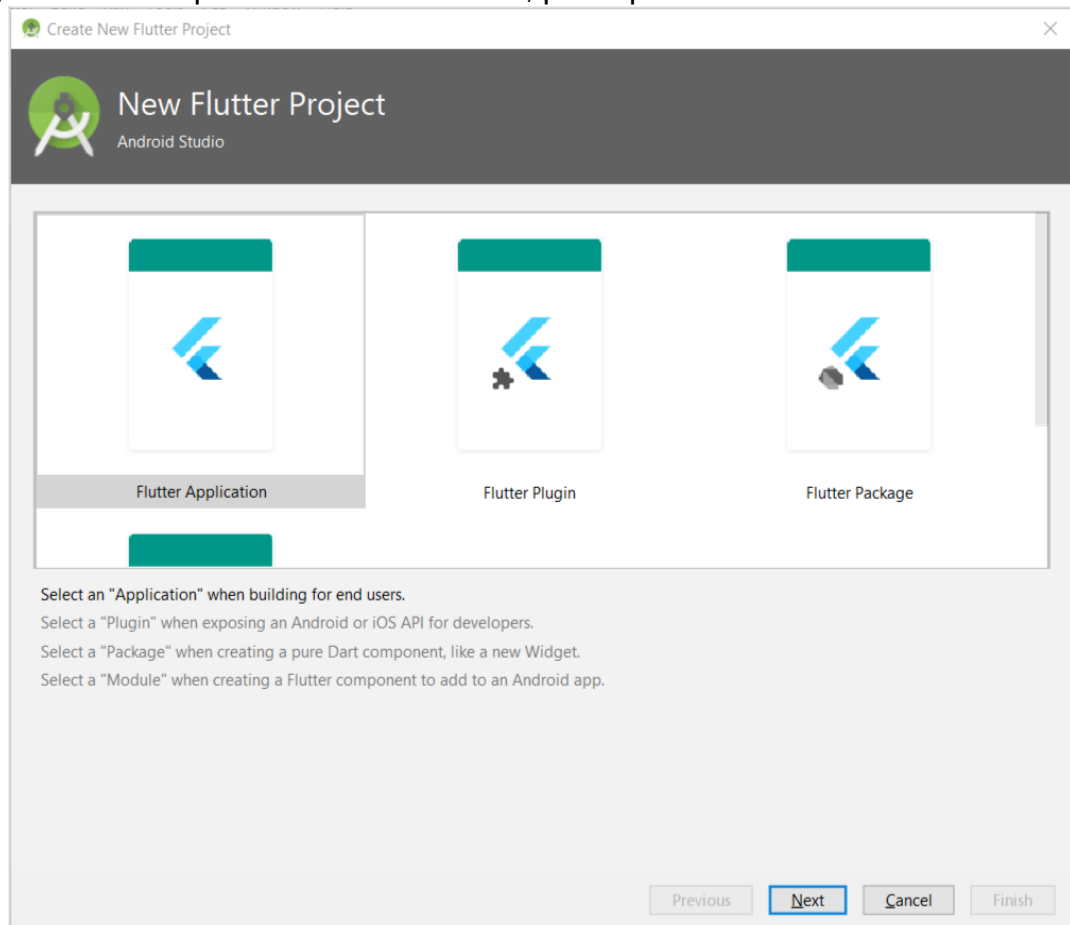
Membuat aplikasi *Flutter* sederhana untuk memahami dasar- dasar pembuatan aplikasi yang berkibar di Android Studio.

Langkah 1: Buka Android Studio

Langkah 2: Buat Proyek Flutter. For this, click **File -> New -> New Flutter Project**

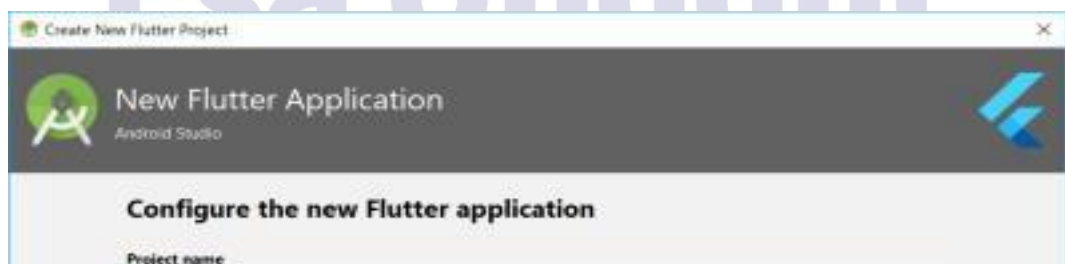


Langkah 3: Pilih Aplikasi Flutter. Untuk ini, pilih Aplikasi **Flutter** dan klik**Next**.



Langkah 4:

- a) Konfigurasi aplikasi seperti di bawah ini dan klikNext.
- b) Project name: hello_app
- c) Flutter SDK Path: <path_to_flutter_sdk>
- d) Project Location: <path_to_project_folder>
- e) Description: Flutter based hello world application

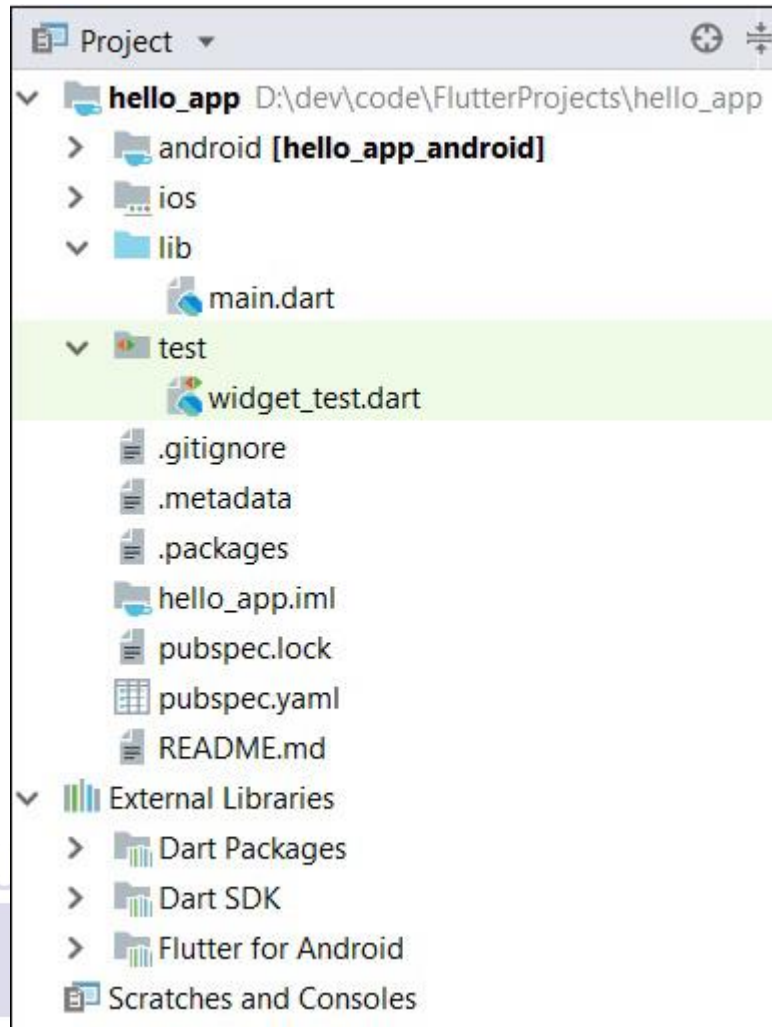


Langkah 5 : Mengonfigurasi Project.

Tetapkan domain perusahaan sebagai flutterapp.tutorialspoint.com dan klik Selesai

Langkah 6 : Masukkan domain Perusahaan.

Android Studio membuat aplikasi yang sepenuhnya berfungsi dengan fungsionalitas minimal. Mari kita periksa struktur aplikasi dan kemudian, mengubah kode untuk melakukan tugas yang dihendaki.



Berbagai komponen struktur aplikasi dijelaskan di bawah ini :

- a) android - Kode sumber yang dibuat secara otomatis untuk membuat aplikasi android
- b) ios - Kode sumber yang dibuat secara otomatis untuk membuat aplikasi ios
- c) lib - Folder utama yang berisi kode Dart yang ditulis menggunakan framework flutter
- d) lib / main.dart - Titik masuk aplikasi Flutter
- e) test - Folder berisi kode Dart untuk menguji aplikasi flutter
- f) test / widget_test.dart - Contoh kode

- g) .gitignore - File kontrol versi Git
- h) .metadata - dibuat secara otomatis oleh alat flutter
- i) .packages - dibuat secara otomatis untuk melacak paket flutter
- j) .iml - file proyek yang digunakan oleh studio Android
- k) pubspec.yaml - Digunakan oleh Pub, pengelola paket Flutter pubspec.lock
- Dibuat secara otomatis oleh pengelola paket Flutter, Pub README.md -
File deskripsi proyek yang ditulis dalam format Penurunan Harga

Langkah 7 - Ganti kode panah di file lib / main.dart dengan kode di bawah ini :

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello World Demo Application',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(title: 'Home page'),
    );
  }
}

class MyHomePage extends StatelessWidget {
  MyHomePage({Key key, this.title}) : super(key: key);
  final String title;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(this.title),
      ),
      body: Center(
        child: Text(
          'Hello World',
        ),
      ),
    );
  }
}
```

Penjelasan Code Program di atas :

1) Baris 1 :

Mengimpor paket flutter, material. Materi adalah paket flutter untuk membuat antarmuka pengguna sesuai dengan pedoman desain Material yang ditentukan oleh Android.

2) Baris 3 :

Ini adalah titik masuk aplikasi Flutter. Memanggil fungsi runApp dan meneruskannya ke objek kelas MyApp. Tujuan dari fungsi runApp adalah untuk memasang widget yang diberikan ke layar.

3) Baris 5 – 17 :

Widget digunakan untuk membuat UI dalam framework flutter. StatelessWidget adalah widget, yang tidak mempertahankan status widget apa pun. MyApp memperluas StatelessWidget dan mengganti metode pembuatannya. Tujuan pembangunan

4) Metode adalah membuat bagian dari UI aplikasi. Di sini, metode build menggunakan MaterialApp, sebuah widget untuk membuat UI level root aplikasi. Ini memiliki tiga properti - judul, tema, dan rumah.

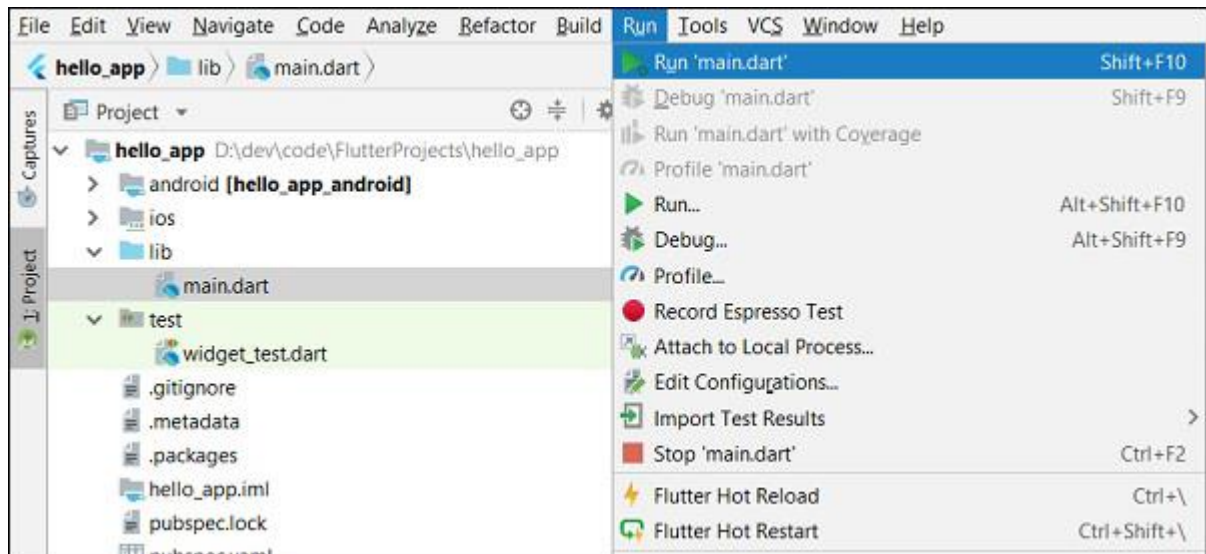
- a. judul adalah judul aplikasi.
- b. theme adalah tema widget. Di sini, kami menetapkan biru sebagai warna keseluruhan aplikasi menggunakan kelas ThemeData dan propertinya, primarySwatch.
- c. home adalah UI bagian dalam aplikasi, yang kami setel widget lain, MyHomePage.

5) Baris 19 - 38: MyHomePage sama dengan MyApp kecuali ia mengembalikan Widget Perancah. Scaffold adalah widget tingkat atas di samping widget MaterialApp yang digunakan untuk membuat desain material yang sesuai dengan UI. Ini memiliki dua properti penting, appBar untuk menampilkan header aplikasi dan body untuk menampilkan konten aplikasi yang sebenarnya. AppBar adalah widget lain untuk merender header aplikasi dan

kami telah menggunakannya di properti appBar. Di properti body, kami telah menggunakan widget Center, yang memusatkannya widget anak. Teks adalah widget paling akhir dan paling dalam untuk menampilkan teks dan ditampilkan di tengah layar.

Langkah 8 :

Sekarang, jalankan aplikasi menggunakan, **Jalankan -> Jalankan main.dart**



Langkah 9 : Akhirnya, output aplikasi adalah sebagai berikut:

